



adc_cpp : architecture d'un solveur AMR / MPI / GPU

Conception template-first pour systèmes hyperbolique-elliptique

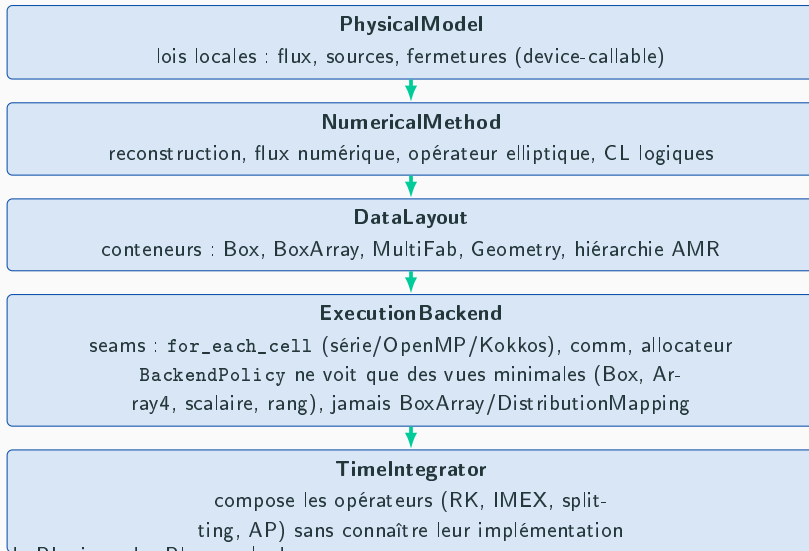
Romain Despoullains | Master CHPS – URCA

1^{er} juin 2026

$$\frac{\partial U}{\partial t} + \nabla \cdot F(U, \text{aux}) = S(U, \text{aux}) \quad \Delta \phi = f(U)$$

Une partie **hyperbolique** (U) et une partie **elliptique** (ϕ), couplées à chaque pas par $\text{aux} = (\phi, \nabla \phi)$.

Architecture en couches



La physique : un concept

Un modèle est un type qui fournit quatre fonctions **locales** :

```
template <class M>
concept PhysicalModel = requires(M m, M::State u, M::Aux a, int dir) {
    { m.flux(u, a, dir) } -> std::same_as<typename M::State>;
    { m.max_wave_speed(u, a, dir)}-> std::convertible_to<Real>;
    { m.source(u, a) } -> std::same_as<typename M::State>;
    { m.elliptic_rhs(u) } -> std::convertible_to<Real>;
};
```

flux, source : les termes de l'équation | max_wave_speed : le pas stable (CFL) |
elliptic_rhs : le couplage Poisson.

Le compilateur vérifie le contrat ; tout type le remplissant est un modèle valide.

Un modèle : le diocotron

```
struct Diocotron {  
    using State = StateVec<1>; using Aux = adc::Aux; // 1 scalaire : n_e  
    static constexpr int n_vars = 1;  
    Real B0 = 1.0, n_i0 = 1.0, alpha = 1.0;  
  
    ADC_HD State flux(const State& u, const Aux& a, int dir) const {  
        Real v = (dir == 0) ? -a.grad_y/B0 : a.grad_x/B0; // v = (E x B)/|B|^2  
        return State{ u[0] * v }; // F = n_e * v  
    }  
    ADC_HD State source(const State&, const Aux&) const { return State{}; }  
    ADC_HD Real elliptic_rhs(const State& u) const { return alpha*(u[0]-n_i0);  
    }  
};
```

L'elliptique : un concept

Même principe pour le solveur de Poisson : un contrat, plusieurs backends.

```
template <class S>
concept EllipticSolver = requires(S s) {
    { s.rhs() } -> std::same_as<MultiFab>; // on écrit f
    { s.phi() } -> std::same_as<MultiFab>; // on lit phi
    s.solve(); // resout Delta phi = f
};
```

Trois implémentations interchangeables :

- GeometricMG (multigrille, général)
- PoissonFFTSolver (spectral direct, périodique)
- DistributedFFTSolver (FFT distribuée, MPI)

Construire son solveur avec adc

Étape 1 : définir son modèle

Un type qui satisfait `PhysicalModel` (advection scalaire, vitesse (a, b)) :

```
struct Advection {  
    using State = adc::StateVec<1>; using Aux = adc::Aux;  
    static constexpr int n_vars = 1;  
    double a = 1.0, b = 0.5; // vitesse d'advection  
    ADC_HD State flux(const State& u, const Aux&, int dir) const {  
        double v = (dir == 0) ? a : b;  
        return State{ v * u[0] }; // F = v u  
    }  
    ADC_HD double max_wave_speed(const State&, const Aux&, int d) const {  
        return (d == 0) ? std::fabs(a) : std::fabs(b); }  
    ADC_HD State source(const State&, const Aux&) const { return State{0.0}; }  
    ADC_HD double elliptic_rhs(const State& u) const { return u[0]; }  
};  
static_assert(adc::PhysicalModel<Advection>); // le compilateur valide le contrat
```


Étape 2 : le faire tourner sur grille uniforme

Le couplage Poisson + transport est encapsulé par `Coupler<Model, Elliptic>` :

```
#include <adc/coupling/coupler.hpp>
#include <adc/elliptic/geometric_mg.hpp>

adc::Geometry geom{dom, 0.0, 1.0, 0.0, 1.0};
adc::Coupler<Diocotron, adc::GeometricMG> sim(model, geom, ba, bc);

// remplir sim.state() (densite initiale), puis :
for (int s = 0; s < nsteps; ++s)
    sim.advance<adc::VanLeer>(dt); // Poisson -> aux -> transport
```

Le coupleur ferme la boucle $\phi \rightarrow \text{aux} \rightarrow \text{avance}$. Pour un modèle pur hyperbolique, on appelle directement `advance_ssprk2`.

Étape 3 : choisir le schéma

Flux et reconstruction sont des **policies** template, pas des branches :

```
sim.advance<adc::VanLeer>(dt); // MUSCL VanLeer  
sim.advance<adc::NoSlope, adc::OncePerStepCoupling>(dt); // ordre 1, Poisson  
                        1x/pas
```

Flux : RusanovFlux, HLLFlux, HLLCFlux.

Reconstruction : NoSlope (ordre 1), Minmod / VanLeer / MC, WENO5Z (ordre 5).

Pour fixer aussi le flux : `compute_face_fluxes<Limiter, NumericalFlux>`. Changer de schéma ne touche jamais le modèle.

Étape 4 : passer en AMR

Même interface, un coupleur de plus : AmrCouplerMP :

```
std::vector<adc::AmrLevelMP> levels(2);
levels[0] = {std::move(Uc), nullptr, dxc, dyc}; // niveau grossier
levels[1] = {std::move(Uf), nullptr, dxc/2, dyc/2}; // un patch fin
adc::AmrCouplerMP<Diocotron> sim(model, geom, ba_coarse, bc, std::move(levels));
auto crit = [&](const adc::ConstArray4& a, int i, int j) { // raffinement
    return a(i, j, 0) > seuil;
};
for (int s = 0; s < nsteps; ++s) {
    if (s % 20 == 0) sim.regrid(crit); // clustering Berger-Rigoutsos
    sim.step(dt); // sous-cyclage + reflux conservatif
}
```

Le critère est à l'utilisateur ; le moteur gère hiérarchie, regrid, reflux.

Étape 5 : choisir le backend (compilation)

Le code source ne change pas. On choisit le backend une fois, à cmake :

```
cmake -B build # série  
cmake -B build -DADC_USE_OPENMP=ON # CPU multi-thread  
cmake -B build -DADC_USE_MPI=ON # distribue  
cmake -B build -DADC_USE_KOKKOS=ON # GPU (CUDA) ou CPU portable
```

Le seam `for_each_cell` se compile en boucle série, `#pragma omp`, ou `Kokkos::parallel_for`. **Passer en MPI ou GPU = un drapeau, pas du code** : les opérateurs (modèle, flux, coupleur) sont inchangés. OpenMP est garanti bit-identique à la série.

Depuis Python

Le solveur compilé est piloté depuis Python (pybind11), sans recompiler :

```
import adc

def run(n, B0): # parametres -> resultat
    cfg = adc.DiocotronConfig()
    cfg.n, cfg.B0 = n, B0
    cfg.ic = adc.DiocotronIC.Band
    sim = adc.DiocotronSolver(cfg)
    for _ in range(500):
        sim.step_cfl(0.4)
    return sim.density() # numpy (n, n)

for n in [128, 256, 512]: # balayage en resolution
    rho = run(n, B0=1.0)
```